

1 3.D - Protocole IP, MTU, Fragmentation

Dans cette partie, nous allons voir les caractéristiques du protocole IP ainsi que le contenu de son entête. Nous allons le faire pour *IPv4* et pour *IPv6*.

Rappelons les caractéristiques du protocole IP ? Il sert à acheminer les paquets sur Internet. Ces paquets sont des paquets IP correspondant au niveau 3 de l'encapsulation du modèle OSI soit la couche réseau. Ils vont de routeur en routeur jusqu'à atteindre leur destination, de proche en proche. On parle de routage de proche en proche sur Internet car chaque routeur n'a besoin de connaître que le routeur suivant pour chaque destination. Rappelons que la topologie d'Internet est inconnue dans sa globalité : les chemins possibles d'une source à une destination sont trop complexes et trop dynamiques pour être connus à l'avance ou être calculés par les routeurs sur l'Internet tout entier.

Les routeurs sont des équipements intermédiaires, qui peuvent être des ordinateurs standards ou des équipements spécifiques, tels que des équipements Cisco ou d'autres équipementiers. Ce qui caractérise un routeur, c'est qu'il dispose de plusieurs cartes réseau qui sont reliées à des réseaux différents. De ce fait, un routeur interconnecte les réseaux auxquels il appartient.

Le protocole IP s'occupe donc de l'acheminement des paquets grâce aux routeurs et à l'adresse IP destination qui figure dans l'entête de chaque paquet IP. Nous avons vu dans une autre partie le contenu des tables de routage. Ces tables de routage indiquent comment acheminer un paquet vers le routeur suivant. On parle aussi de **datagramme** quand on parle d'un paquet IP, ce qui fait référence au fait que le protocole n'apporte aucune garantie sur l'acheminement des paquets. C'est aussi pourquoi on parle du mode **best effort**. Le protocole IP fait au mieux, ce qui signifie que si un paquet est perdu, il ne sera pas capable de le détecter et de le retransmettre. Cela est dû au fait que le protocole IP doit être rapide. C'est aussi pour cela qu'il a **peu de fonctionnalités**. Le principe est le suivant. Chaque paquet IP va de routeur en routeur jusqu'à atteindre sa destination. Pour chaque paquet IP entrant, le routeur extrait l'adresse IP destination se trouvant dans l'entête IP et cherche dans sa table de routage l'adresse IP du routeur suivant pour atteindre cette destination. Il fabrique alors une nouvelle trame à destination du routeur suivant contenant le paquet IP. Le routeur suivant fera la même chose et ainsi de suite jusqu'à atteindre la destination finale du paquet.

L'algorithme du routeur est simple et optimisé pour aller le plus vite possible. Dans le protocole IP, il n'y a pas beaucoup d'intelligence, pas d'algorithmes compliqués, simplement une recherche de la bonne ligne dans la table de routage. C'est un protocole qui est **simple** et **robuste**. Pourquoi robuste ? Cela fait référence au routage dynamique dont on a parlé dans une autre partie. Le routage est dit dynamique si le contenu des tables de routage s'adapte automatiquement aux pannes éventuelles qui peuvent survenir dans le réseau (coupure d'électricité, panne d'une carte réseau, lien qui se débranche, arrivée d'un nouveau routeur, etc). Cette mise à jour automatique des tables de routage nécessite la configuration et l'usage de protocoles de routage dynamique comme RIP ou OSPF qui consistent en l'échange régulier (par exemple toutes les 30 secondes) d'informations entre routeurs sur l'état du réseau.

Pour terminer sur les caractéristiques du protocole IP, il est en **mode non connecté**. Cela signifie que chaque paquet est envoyé au routeur suivant sans lui demander l'autorisation au préalable. Le paquet est tout simplement transmis sur le lien qui permet d'atteindre le routeur suivant sans savoir si ce paquet sera recevable. Si le routeur suivant est tombé en panne ou s'il est surchargé, c'est-à-dire qu'il n'a plus de place pour stocker les paquets qui rentrent, le paquet sera perdu. Le mode non connecté va de pair avec le mode *best-effort* : on ne demande pas au routeur s'il est en vie avant de lui envoyer le paquet.

Voilà pour les caractéristiques du protocole IP. On a donc un protocole qui est là pour acheminer les paquets jusqu'à leur destination en faisant au mieux.

Rappelons que ce protocole a un autre rôle moins connu qui est **la fragmentation** des paquets IP, c'est-à-dire l'adaptation de leur taille à la **MTU** (Unité de Transmission Maximale) du réseau physique traversé. Quand on traverse un réseau, il y a une taille maximale pour les trames qui vont traverser ce réseau. La MTU est donc la taille maximale qu'un paquet IP peut avoir pour être mis dans la trame. En effet, un paquet IP (de niveau 3) est encapsulé dans une trame (de niveau 2). Cette trame a une taille maximale qu'il faut respecter. Donc, si le paquet IP est trop grand, l'émetteur du paquet (équipement final ou routeur

intermédiaire) doit découper le paquet en plusieurs morceaux, chaque morceau ou fragment devenant un nouveau paquet IP admissible dans une trame.

Nous verrons par la suite comment fonctionne ce découpage des paquets IP qui sont trop grands pour respecter la taille maximale des trames.

1.1 Protocole et entête IPv4

Nous pouvons maintenant décrire l'entête IPv4, c'est-à-dire le contenu de l'entête du protocole qui va permettre de réaliser l'acheminement des paquets jusqu'à leur destination, mais aussi la fragmentation. Le format de l'entête IPv6 est différent et sera présenté plus loin. Quand on représente un entête, on le fait toujours par des lignes qui font 4 octets (32 bits). C'est parce que l'adresse IPv4 est codée sur 32 bits. Cette représentation par ligne de 4 octets est effective pour tous les protocoles.

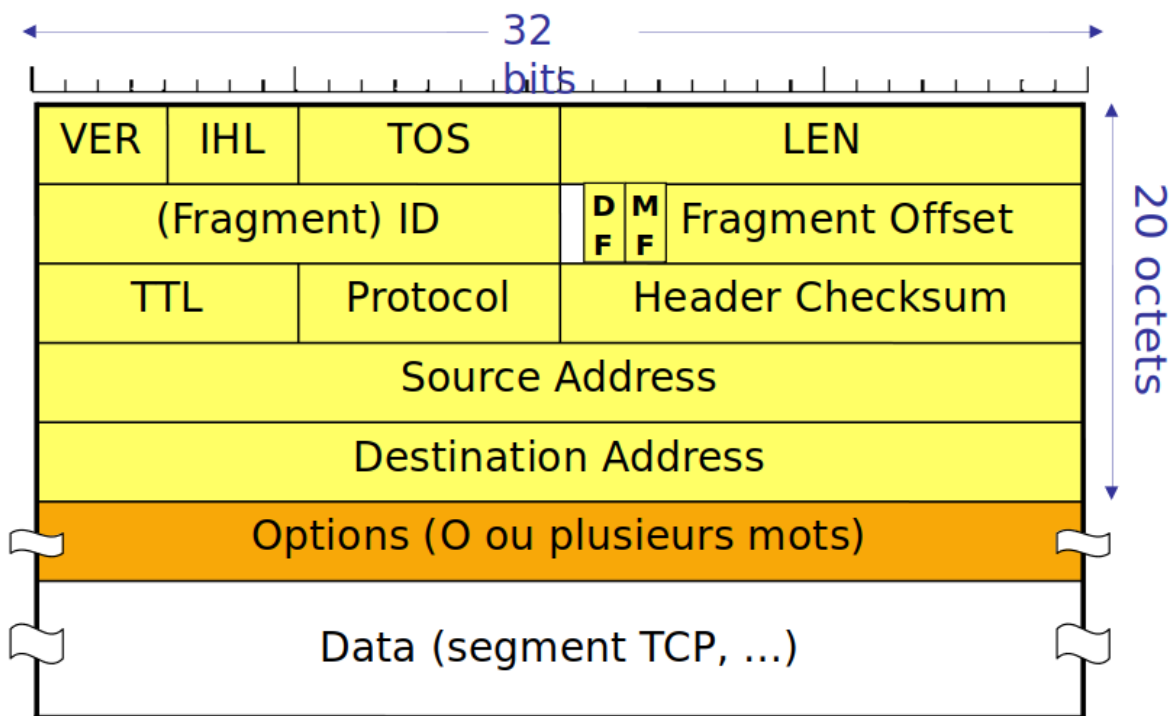


Figure 1: Entête IPv4

L'entête IP fait 5 lignes de 32 bits chacune, c'est-à-dire quatre octets, et donc elle fait 20 octets. Le protocole a prévu la possibilité d'ajouter des entêtes optionnels, représentés en orange mais ils ne sont pas ou peu utilisés sur Internet.

Alors voyons précisément de quoi est constitué cet entête. Tout d'abord, la **version du protocole** (VER) qui se trouve codée sur les quatre premiers bits (un demi-octet) de l'entête IPv4 et IPv6. La valeur de ce champ est donc 4 pour IPv4 et 6 pour IPv6. Cela permet aux routeurs qui reçoivent le paquet de savoir s'il s'agit d'un paquet IP version quatre ou six.

Juste après figure le champ **Internet Header Length** (IHL) qui permet de donner la taille de l'entête. Il stocke, sur quatre bits également, le nombre de lignes qui constituent l'entête. Pour une entête IPv4 sans option, la valeur est donc 5. Ainsi, tous les paquets IPv4 commencent par l'octet 45 en hexadécimal.

Vous avez ensuite un octet pour le champ **Type of Service** (TOS). Il permet de faire de la qualité de service en donnant une priorité différentes dans le traitement de certains paquets par rapport à d'autres.

Par exemple, on pourrait décider que les flux vidéos sont prioritaires sur Internet ! Une autre politique consisterait à définir la priorité en fonction du prix d'abonnement payé par l'utilisateur. Fort heureusement, ce n'est pas le cas sur l'Internet d'aujourd'hui : tous les paquets sont traités à égalité. De ce fait, le champ TOS est toujours à zéro car on considère que tous les paquets ont la même priorité.

Le champ suivant sur deux octets est LEN pour **Length**. Il est très important car il donne la taille totale du paquet, c'est à dire la taille de l'entête plus la taille du contenu. Il est codé sur deux octets (seize bits). De ce fait, la taille maximale d'un paquet IP est de 65 535 octets. Le contenu du paquet correspond à la section DATA qui suit l'entête. Que peut être le contenu d'un paquet IP ? Soit un flux TCP pour des échanges entre par exemple un serveur Web et un navigateur Web. Soit un paquet UDP (on parle aussi de datagramme UDP) pour une application qui utilise UDP, comme par exemple le service DNS. Soit une requête ou réponse ICMP correspondant généralement à l'usage de la commande *ping* ou *traceroute*. Cela permet de tester le bon fonctionnement d'Internet entre une adresse IP source et destination. Ces commandes peuvent aussi afficher le temps aller-retour et les routeurs traversés entre la source et la destination. Il y a donc trois possibilités dans **Data** qui correspond au contenu du paquet IP comme on l'a vu sur l'architecture TCP/IP : TCP, UDP ou ICMP.

La deuxième ligne de l'entête permet de faire la fragmentation, c'est-à-dire le découpage d'un paquet IP trop gros relativement à la taille maximale de la trame, en plusieurs morceaux appelés fragments, chaque fragment devenant lui-même un paquet IP. Cette ligne permet de repérer les différents morceaux pour reconstituer le paquet IP d'origine sur la destination finale. Le champ **ID** contient le numéro du paquet. Il est distinct d'un paquet à l'autre mais il est identique pour tous les fragments d'un même paquet d'origine. Il permet de remettre ensemble tous les morceaux d'un même paquet, puisqu'ils auront tous le même ID. Les trois champs suivants sont aussi relatifs à la fragmentation. Le champ **Don't fragment** (DF) indique s'il est autorisé ou non de fragmenter ce paquet ; il s'agit d'un choix de l'émetteur. Le champ **More fragment** (MF) indique s'il y a d'autres fragments qui suivent ou si c'est le dernier fragment. Enfin, le champ **Fragment offset** indique la position du fragment dans le paquet d'origine. Si les deux derniers octets de cette deuxième ligne de l'entête sont à zéro, cela signifie que ce paquet n'a pas été fragmenté. Nous détaillerons plus loin un exemple de fragmentation d'un paquet IP.

La troisième ligne de l'entête commence par le champ **Time to Live** (TTL). Il indique la durée de vie du paquet, le temps restant à vivre, c'est-à-dire le nombre de routeurs qu'il peut encore traverser. En général, la valeur initiale du TTL au départ est de 32. Si un paquet traverse plus de 32 routeurs, cela signifie qu'il est perdu ou qu'il tourne en boucle dans le réseau à cause d'une erreur dans les tables de routage. Les routeurs éliminent les paquets qui arrivent avec un TTL à zéro.

Le champ suivant, sur un octet, est le champ **Protocol**. Ce champ stocke le numéro du protocole qui est contenu dans le paquet. La numérotation de ces protocoles est fixée par l'[IANA](#). Les 3 principaux protocoles encapsulés dans IP sont TCP (Protocol=6), UDP (Protocol=17) et ICMP (Protocol=1).

La troisième ligne se termine par un "petit" **checksum** sur l'entête. Il sert à vérifier qu'il n'y a pas d'erreur dans l'entête du paquet IP. Cela permet d'éviter de continuer à router un paquet qui a subi une erreur sur les adresses IP pendant le transfert précédent. Ce calcul de checksum est très rapide, il est fait seulement sur les 20 octets d'entête et ne permet pas de détecter des erreurs dans le contenu du paquet.

Enfin, la quatrième et cinquième lignes stockent l'**adresse IP source** et l'**adresse IP destination** sur 4 octets chacune. Et donc c'est bien cette dernière qui va être utilisée pour la recherche dans la table de routage du prochain saut.

1.2 Fragmentation IPv4

Nous avons décrit précédemment le principe de la fragmentation et la ligne correspondante dans l'entête IPv4 (deuxième ligne). Il s'agit de découper en plusieurs morceaux/fragments un paquet IP qui serait trop gros relativement à la MTU de la trame, c'est-à-dire la taille maximale des données qui peuvent être encapsulées dans la trame. La MTU dépend du type de la liaison qui va être empruntée. Par exemple, la MTU d'une liaison Ethernet est de 1500 octets. La fragmentation est réalisée par l'émetteur du paquet. Il peut s'agir soit de l'ordinateur final hébergeant le client ou le serveur, soit d'un routeur intermédiaire.

Lorsqu'un paquet est découpé en plusieurs fragments pour traverser une liaison, chaque fragment est lui-même un nouveau paquet IP qui va cheminer jusqu'à sa destination finale. Une fois que tous les fragments sont arrivés à la destination finale, ils sont rassemblés pour reformer le paquet IP d'origine.

Sur Internet, il est possible d'avoir des **fragmentations multiples** : un paquet peut subir une première fragmentation lors de la traversée d'un premier réseau puis l'un ou plusieurs de ces fragments pourront à leur tour subir une fragmentation lors de la traversée d'un autre réseau ayant une MTU plus petite que le premier réseau. Comme la fragmentation peut être multiple et que les fragments peuvent emprunter des routes différentes (du fait du routage dynamique par exemple), le réassemblage des fragments n'est réalisé qu'au niveau de la destination finale.

Voyons maintenant un exemple de fragmentation.

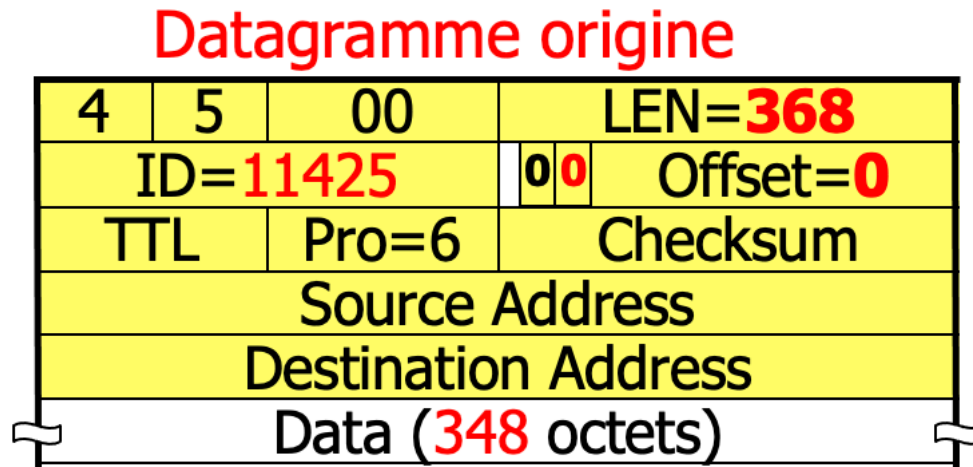


Figure 2: Exemple de paquet trop gros à fragmenter

Vous avez ci-dessus, le paquet IP d'origine avec son entête IPv4 et un contenu de 348 octets. Ce contenu correspond à TCP puisque le champ Protocol a la valeur 6. Les différents champs de l'entête sont indiqués ici avec leur valeur d'origine avant d'être fragmenté. Le paquet a une longueur de 368 octets, comme indiqué dans le champ LEN : 20 octets pour l'entête + 348 pour les données.

Examinons la deuxième ligne de l'entête. Pour un paquet qui n'est pas fragmenté, vous verrez toujours l'offset à zéro et les deux bits précédents à zéro également. En effet, le bit MF=0 signifie qu'il s'agit du dernier fragment et l'offset à 0 signifie qu'il s'agit du premier fragment. Il s'agit donc du premier et dernier fragment d'un même paquet donc du paquet lui-même. L'ID d'une valeur de 11425 correspond au numéro identifiant le paquet IP lors de sa création.

Maintenant, on souhaite envoyer ce paquet sur un réseau qui a une MTU de 128 octets. Comme notre paquet d'origine a une longueur de 368 octets supérieure à 128, il va falloir le fragmenter.

Intuitivement, on pourrait se dire qu'il suffit de fragmenter ce paquet en 3 fragments puisque $3 \times 128 = 384$ et que 384 est supérieur à 368. Mais cela n'est pas suffisant car pour chaque fragment il faut ajouter (dupliquer d'une certaine manière) l'entête IP de 20 octets. C'est pourquoi, 4 fragments vont être nécessaires. Ils sont détaillés ci-dessous.

Chaque fragment est un paquet IP qui doit avoir une taille maximale de 128 octets. Si la MTU est de 128 octets, il faut faire des paquets IP qui ont 20 octets d'entête et au maximum 108 octets de données. Ici, on a mis seulement 104 octets de données et on va expliquer pourquoi. Vous voyez aussi que tous les fragments sont proches de la taille maximale, sauf le dernier, c'est-à-dire qu'on ne cherche pas à découper en faisant des tailles égales de tous les fragments. On remplit les premiers fragments au maximum et le dernier fragment prend ce qu'il reste : $348 - 3 \times 104$ soit 36 octets.

L'entête IP se reproduit dans chaque fragment en conservant toutes ses lignes identiques, sauf la deuxième

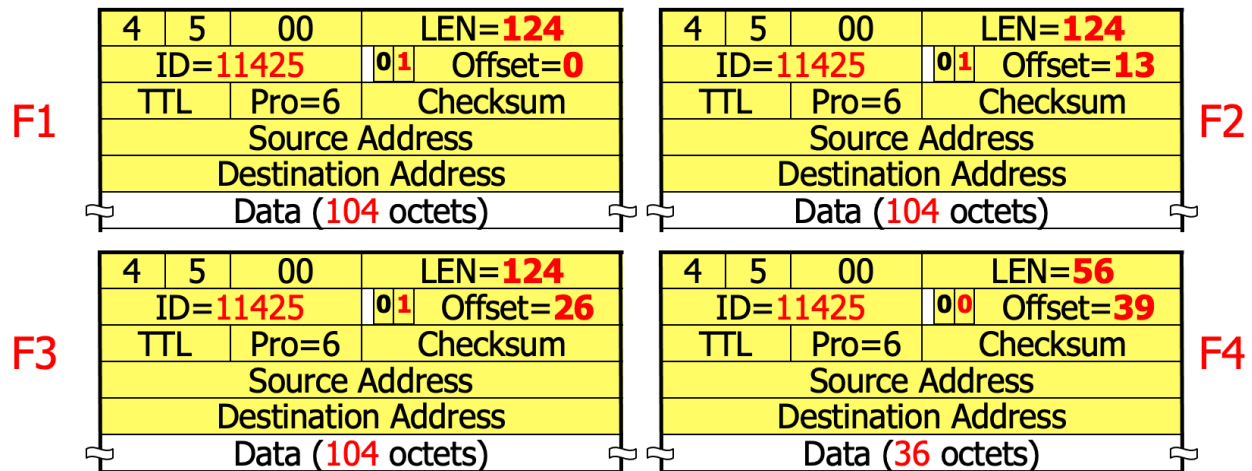


Figure 3: Même paquet fragmenté en 4 paquets

ligne qui concerne la fragmentation, et bien sûr le champ LEN qui lui aussi varie. Le champ LEN indique bien $20+104=124$ pour les 3 premiers fragments et $20+36=56$ pour le dernier fragment. Le champ ID est le même pour tous les fragments car c'est l'identifiant du paquet d'origine.

Maintenant, examinons les deux octets à droite sur la deuxième ligne : le champ *fragment offset*, ainsi que les bits *don't fragment* et *more fragment*. Le bit *don't fragment* est à 0 pour tous les fragments, ce qui signifie que nous avons le droit de refragmenter ces fragments. Le champ *more fragment* est à 1 pour tous les fragments sauf le dernier, ce qui est le comportement attendu. On peut également remarquer qu'il y a un bit resté blanc (juste avant le bit "don't fragment") : ce bit n'est pas utilisé, il est réservé pour une utilisation future en cas de besoin. En effet, lorsqu'on conçoit un protocole, on essaie de le rendre évolutif. C'est aussi la raison d'être des options dans l'entête IP.

Enfin, il y a le champ *fragment offset*. Il permet de numérotiser les différents fragments en indiquant le rang dans le paquet d'origine du premier octet transporté, c'est-à-dire l'endroit où il se trouve. Rappelons-nous que les fragments peuvent utiliser des routes différentes et donc arriver dans le désordre. Ce champ nous permettra de remettre tous les fragments dans le bon ordre. On peut remarquer également que ce système de numérotation permet la fragmentation multiple.

Le premier fragment a l'offset zéro. Le deuxième a un offset de 13 car $13*8=104$. En effet, l'offset stocke le rang dans le paquet d'origine du premier octet transporté divisé par 8. Le nombre d'octets transportés dans un fragment est donc nécessairement un multiple de 8 octets sauf dans le dernier fragment qui contient les octets restants dans les données. Ici, nous avons des fragments de longueur 124 et non 128 car 108 n'est pas un multiple de 8 alors que 104 l'est. Chaque fragment stocke donc 104 octets de données et l'offset s'incrémente de 13 pour chaque fragment. Par exemple, le deuxième fragment a un offset de 26 ce qui correspond à un décalage de $26*8=208$. Le premier octet transporté dans ce fragment est le 208ème dans le paquet d'origine.

Pourquoi l'offset est-il divisé par 8 ? Tout simplement parce que l'offset est sur 13 bits et qu'un paquet IP peut avoir une taille allant jusqu'à 65535 octets. En imaginant le pire cas où chaque fragment ne contiendrait qu'un seul octet, l'offset devrait pouvoir contenir une valeur maximale de 65535 ce qui n'est pas possible sur 13 bits. La valeur maximale sur 13 bits est 8191. Pour combler les 3 bits manquants, il suffit de considérer que l'offset stocké doit être multiplié par 8 pour obtenir l'offset réel. Ainsi, si l'offset stocke la valeur 8191, cela signifie que le premier octet transporté dans le fragment est le 65528ème dans le paquet d'origine.

1.3 Protocole et entête IPv6

Voyons maintenant les caractéristiques du protocole IP version 6 (IPv6). Pourquoi IPv6 est-il apparu après IPv4 ? Nous l'avons abordé dans les séquences précédentes. La raison d'être d'IPv6 est avant tout le manque

d'adresses IPv4. En effet, il y a environ 4 milliards d'adresses IPv4 (2^{32}) alors qu'il y a à ce jour plus de 16 milliards d'équipements sur Internet nécessitant une adresse IP. Les adresses IPv6 sont quatre fois plus grandes que les adresses IPv4 (128 bits au lieu de 32 bits) ce qui permet d'avoir un nombre d'adresses suffisamment important pour l'ensemble de la planète, même avec plusieurs milliards d'habitants. Pour faire face à cette pénurie d'adresses avec IPv4, on utilise également le NAT (*Network Address Translation*) et l'adressage sans classe, le CIDR, qui permet de faire des réseaux de taille la plus proche possible des besoins de l'organisation.

IPv4 et IPv6 cohabitent depuis de nombreuses années, et encore sans doute pour plusieurs années à venir, du fait qu'il n'est pas possible, du jour au lendemain, de remplacer un protocole par un autre. Il y a des zones d'Internet qui sont en IPv4, d'autres en IPv6, d'autres qui supportent les deux types d'adresses. Tout ça cohabite très bien grâce à des techniques qui nous permettent de remplacer des adresses IPv4 par des adresses IPv6 et vice-versa.

IPv6 a commencé à être normalisé dans les années 1990. En plus de fournir un nombre d'adresses au-delà des besoins de l'humanité, il permet d'optimiser un certain nombre de caractéristiques du protocole IPv4 pour accélérer le traitement des paquets dans les routeurs. Pour cela, il faut réduire la taille des tables de routage et faire en sorte que le protocole soit le plus léger possible. IPv6 améliore également la sécurité et diminue le nombre de diffusions, en privilégiant les adresses multicast qui désignent des groupes d'utilisateurs. Il est également un peu plus évolutif que IPv4, en pouvant chaîner plusieurs entêtes de manière évolutive.

IPv6 est compatible avec les autres protocoles de l'architecture TCP/IP. Il a donc des adresses qui sont sur 16 octets au lieu de 4 dans le cas d'IPv4. De ce fait, l'entête fait 40 octets au lieu de 20 octets en IPv4. Dans le but d'accélérer le traitement des paquets, l'entête est simplifiée : elle comporte seulement sept champs au lieu de treize en IPv4. Il y a en gros deux principes qui permettent d'alléger le traitement des paquets IPv6 par les routeurs. C'est d'abord le fait de considérer qu'il n'y a plus besoin de fragmentation, car il est possible de faire de la découverte de la MTU sur un chemin, depuis la source jusqu'à la destination, et donc de connaître à l'avance depuis l'émetteur du paquet, la taille maximale des trames sur le chemin qu'emprunteront les paquets. Ça permet donc de faire de la fragmentation, toujours si nécessaire, au niveau de l'émetteur, mais pas à l'intérieur du réseau et donc pas sur les routeurs. Du coup, la fragmentation va se faire de manière optionnelle et seulement quand elle est nécessaire. Et les champs de fragmentation vont donc disparaître de l'entête IPv6. Une autre chose pour accélérer le traitement des paquets par les routeurs est le fait qu'on va déporter le contrôle de la somme de contrôle dans la couche supérieure au niveau de TCP ou UDP. Ainsi, le routeur ne calcule plus le checksum pour chaque paquet qui le traverse.

Les champs qui disparaissent de l'entête IPv6 par rapport à IPv4 sont ceux qui concernent la fragmentation, le header checksum et le IHL (Internet Header Length) puisque en IPv6 les options éventuelles sont chaînées les unes derrière les autres, ce qui revient à faire des options mais de manière beaucoup plus souple et surtout optionnelle au sens où on a par défaut un entête qui fera toujours 40 octets.

Certains champs de l'entête IPv6 ont été renommés. TOS devient **Class of Traffic**. C'est toujours pour faire de la qualité de service et prioriser certains trafics par rapport à d'autres. Le champ **Flow label** permet d'identifier tous les paquets qui appartiennent au même flux, c'est-à-dire qui vont de la même source à la même destination pour une même application. L'objectif est que les routeurs utilisent ce *flow label* pour prendre des décisions d'acheminement plus rapidement car elles seront identiques pour tous les paquets d'un même flux.

Le champ LEN a été remplacé par le champ **Payload length**, c'est-à-dire la taille du contenu du paquet alors que le champ LEN contenait la taille du paquet total. Le champ TTL a été renommé en **Hop Limit**. Enfin, le champ **Next header** remplace le champ protocole, puisqu'en fait il désigne le contenu du paquet et peut être soit de nouveau de l'IPv6, soit éventuellement de l'IPv4, soit les protocoles qui sont encapsulés dans IP, à savoir ICMP, TCP, UDP principalement.

Voici un exemple de capture d'un paquet IPv6 avec *Wireshark* :

Cette trame contient une requête HTTP de type GET qui est encapsulée dans TCP. Le champ *next header* indique bien TCP. Ce paquet a une taille de 260 octets auxquels on ajoute les 40 octets d'entête IPv6. Ça fait donc un paquet IPv6 de 300 octets.

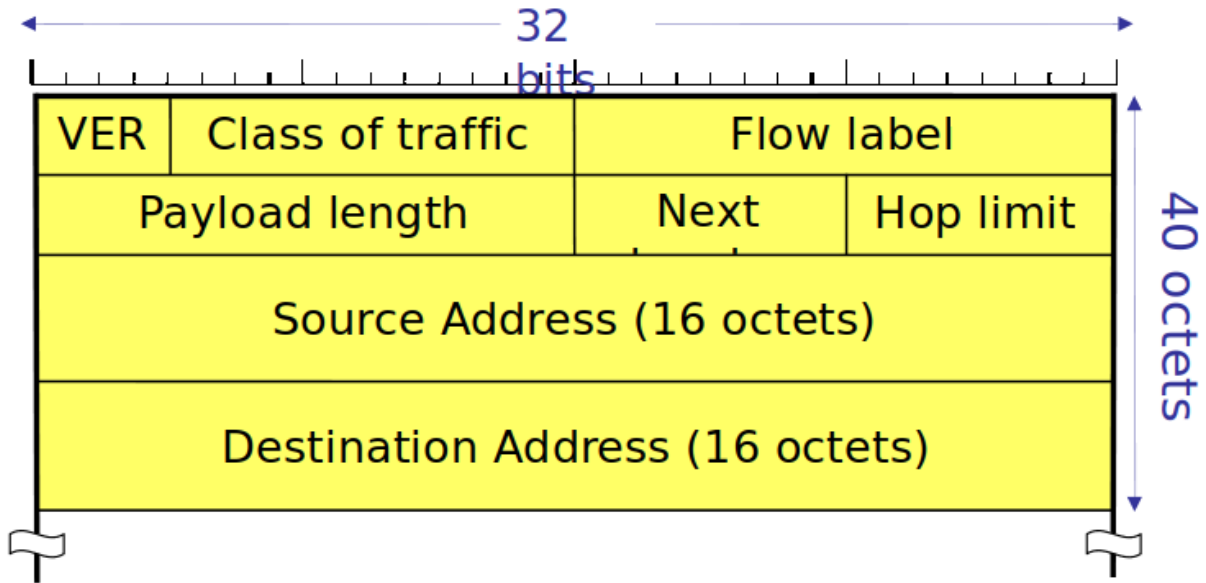


Figure 4: entête IPv6

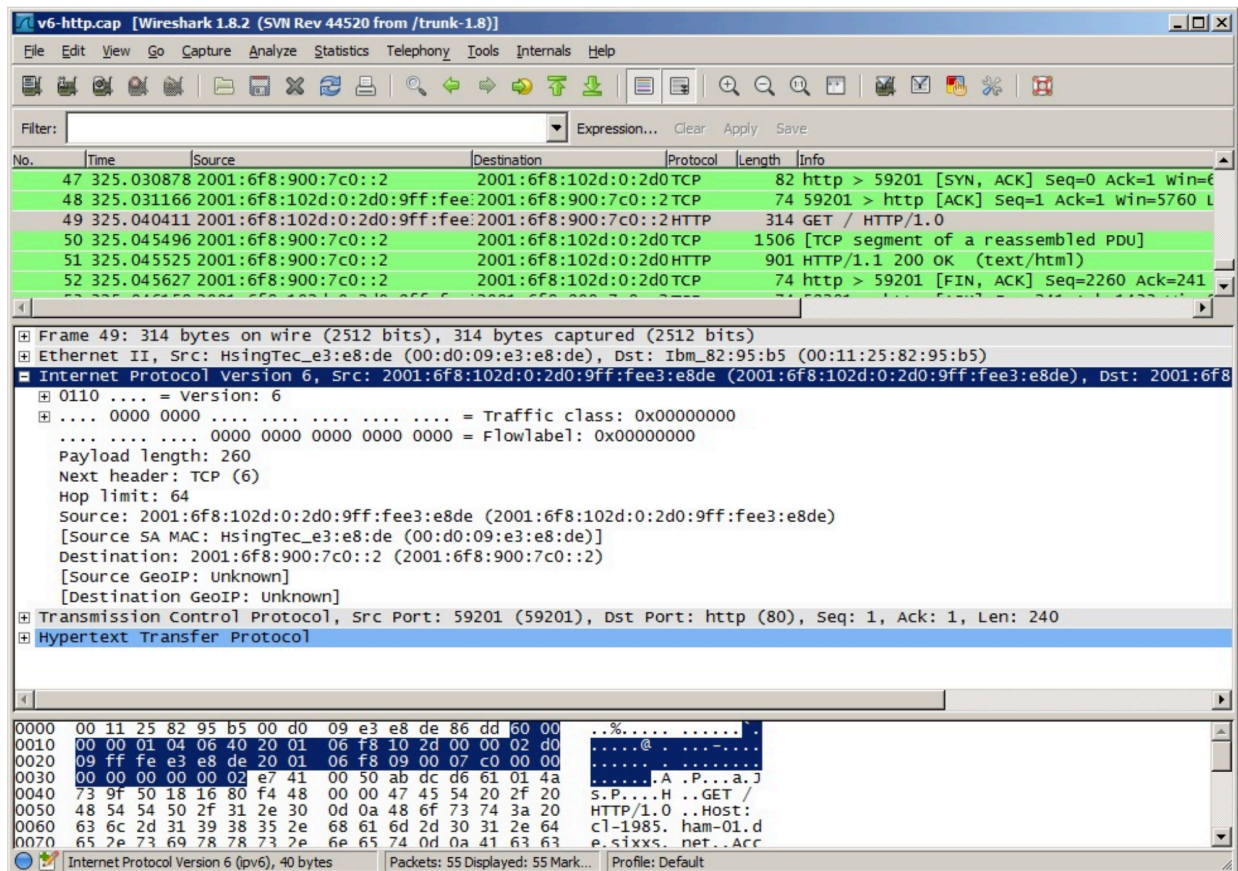


Figure 5: Exemple de paquet IPv6

L'adresse source est 2001:6f8:102d:0:2d0:9ff:fee3:e8de et l'adresse destination est 2001:6f8:900:7c0::2.

Pour finir, voici un exemple de chainage de plusieurs options dans l'entête :

Le champ *Next Header* (NH) :

0 : option "Hop-by-Hop"
 4 : IPv4
 6 : TCP
 17 : UDP
 43 : option "Routing Header"
 44 : option "Fragment Header"
 45 : Interdomain Routing Protocol
 46 : RSVP
 50 : option "Encapsulation Security Payload" (IPsec)
 51 : option "Authentication Header" (IPsec)
 58 : ICMP
 59 : No next header
 60 : option "Destination Options Header"

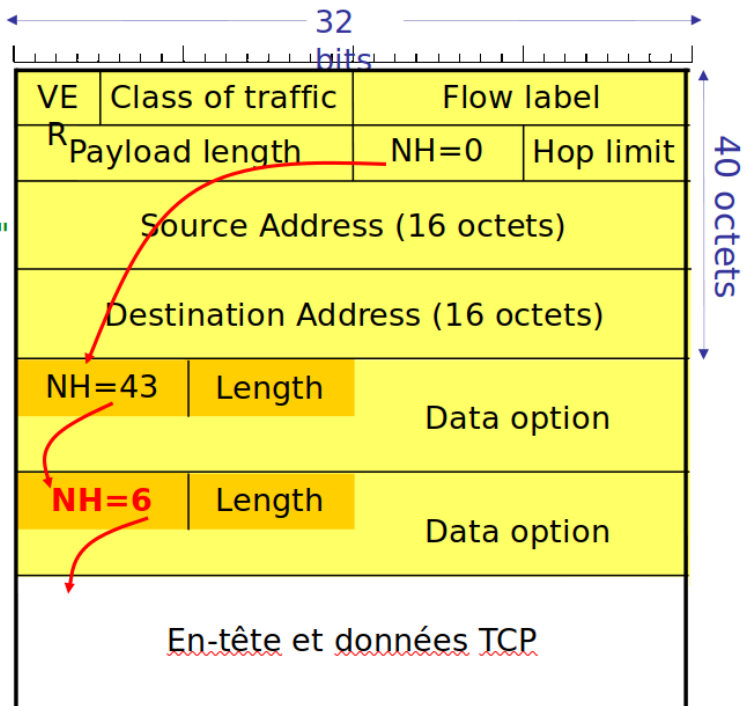


Figure 6: Chaînage des options

Ceci permet d'avoir, dans cet exemple, un en-tête IPv6 suivie d'un en-tête de type « Hop by Hop », qui elle-même est suivie d'un en-tête de type « Routing Header », qui elle-même est suivie d'un en-tête de type TCP. Voilà comment fonctionne le chaînage des différentes options possibles. Elles permettent d'être encapsulées dans des paquets IPv6. Chaque entête doit préciser sa taille dans un champ qui suit le numéro de l'entête.